Informe: algoritmos de clasificación en el diagnóstico de cáncer de mama

Jamer Camilo Bohorquez Cruz

Facultad de ingeniería;Univerisdad de Cundinamarca

604031: INTRODUCCION A MACHINE LEARNING

Mónica María Fonseca Vigoya

16 de octubre de 2026

Resumen

Este informe analiza, con enfoque académico, el comportamiento práctico de cinco algoritmos de clasificación (Árbol de Decisión, Random Forest, SVM, K-NN y Naive Bayes Gaussiano) aplicados al conjunto *Breast Cancer Wisconsin (Diagnostic)*, disponible en *scikit-learn*. Este conjunto tiene 569 observaciones y 30 variables predictoras, con una clase maligna (212 casos) menos numerosa que la benigna (357 casos). El objetivo no fue declarar un ganador, sino observar cómo aprende y decide cada algoritmo y qué tipo de errores comete.

Para que la comparación fuera objetiva, los cinco modelos usaron la misma partición 80/20 (random_state=42, stratify=y), un conjunto de prueba que no se modificó, escalamiento solo en los algoritmos basados en distancias (SVM y K-NN, mediante *pipeline* para evitar fuga de datos) y las mismas métricas: *Accuracy*, *Precision*, *Recall*, *F1-score* y matriz de confusión. La variable objetivo se recodificó para que 1 represente el caso maligno.

El SVM obtuvo el mejor desempeño (*Accuracy* 0.9737, 3 falsos negativos), seguido de Random Forest (0.9649, 4 falsos negativos), K-NN (0.9561, 4 falsos negativos y 1 falso positivo), Naive Bayes (0.9386, 7 falsos negativos) y el Árbol de Decisión (0.9123, 8 falsos negativos y 2 falsos positivos). Como en un diagnóstico el error más costoso es no detectar un tumor maligno, el análisis priorizó el *Recall* de la clase maligna y la matriz de confusión sobre el *Accuracy*. Se observó además el compromiso entre desempeño e interpretabilidad: el Árbol de Decisión es el más fácil de explicar, pero fue el que más casos malignos dejó pasar, mientras que el SVM, con la frontera curva del núcleo rbf, es el más difícil de comunicar a personal no técnico.

Las conclusiones provienen de una única partición de prueba de 114 registros y de hiperparámetros sin ajustar. La diferencia entre SVM y Random Forest es de un solo caso, por lo que no es concluyente. Los modelos tienen fines académicos y no sustituyen el criterio clínico.

Abstract

This report presents an academic analysis of the practical behavior of five classification algorithms (Decision Tree, Random Forest, SVM, K-NN, and Gaussian Naive Bayes) applied to the *Breast Cancer Wisconsin (Diagnostic)* dataset, available in *scikit-learn*. The dataset contains 569 observations and 30 predictor variables, with the malignant class (212 cases) less numerous than the benign class (357 cases). The goal was not to declare a winner, but to observe how each algorithm learns and makes decisions, and what kinds of errors it makes.

To ensure an objective comparison, all five models used the same 80/20 split (random_state=42, stratify=y), an untouched test set, scaling only for distance-based algorithms (SVM and K-NN, applied through a *pipeline* to prevent data leakage), and the same metrics: *Accuracy*, *Precision*, *Recall*, *F1-score*, and the confusion matrix. The target variable was recoded so that 1 represents the malignant case.

The SVM achieved the best performance (*Accuracy* 0.9737, 3 false negatives), followed by Random Forest (0.9649, 4 false negatives), K-NN (0.9561, 4 false negatives and 1 false positive), Naive Bayes (0.9386, 7 false negatives), and the Decision Tree (0.9123, 8 false negatives and 2 false positives). Because the costliest error in a diagnostic setting is failing to detect a malignant tumor, the analysis prioritized the *Recall* of the malignant class and the confusion matrix over *Accuracy*. A trade-off between performance and interpretability was also observed: the Decision Tree is the easiest to explain but missed the most malignant cases, while

the SVM, with the curved decision boundary of its rbf kernel, is the hardest to communicate to non-technical staff.

The conclusions derive from a single test split of 114 records and untuned hyperparameters. The difference between the SVM and Random Forest is a single case, so it is not conclusive. The models are for academic purposes only and do not replace clinical judgment.

Keywords: classification, machine learning, confusion matrix, Recall, breast cancer, scikit-learn

Contenido

Resumen 2

Abstract 3

Introducción 9

Caso de trabajo 11

Misma división de datos 13

Proporción 80/20 con partición estratificada. 13

Conjunto de prueba intocable. 13

Escalamiento solo donde es necesario. 13

Mismas métricas para todos. 14

No elegir el modelo solo por Accuracy. 14

Preparación de los datos 15

Importación de pandas y data set 15

Carga del dataset en un Dataframe 16

Conversión la clase positiva en "Maligno" 17

División de datos 18

Función común de evaluación 20

Accuracy 21

Precision 21

Recall 21

F1-score 21

Matriz de confusión 21

Implementación de los clasificadores 22

Árbol de Decisión (*Decision Tree*) 22

Random Forest 23

Máquina de Soporte Vectorial (Support Vector Machine, SVM) 25

K-Vecinos más Cercanos (*K-Nearest Neighbors*, K-NN) 26

Naive Bayes Gaussiano (GaussianNB) 27

Comparación para la toma de decisiones 28

Preguntas de análisis 29

¿Qué modelo obtuvo la mayor Accuracy? 29

¿Qué modelo obtuvo el mayor Recall para la clase maligna? 29

¿Qué modelo produjo menos falsos negativos? 30

¿Coincide el modelo con mayor Accuracy con el modelo que seleccionarías? Explica. 30

Si el costo de no detectar un caso maligno fuera especialmente alto, ¿qué métrica priorizarías y por qué? 30

¿Cuál de los modelos es más sencillo de explicar a una persona no técnica? 31

¿Qué diferencias observaste entre los algoritmos que requieren escalamiento y los que no? 31

Selecciona un modelo final y justifica tu decisión usando al menos dos métricas y una consideración del contexto 32

Respuestas de preguntas 33

Preparación del caso y datos 33

¿Cuántas observaciones y variables contiene el conjunto de datos? 33

¿La variable objetivo está balanceada? Explica brevemente lo que observas. 33

¿Por qué es importante definir con claridad qué representa la clase positiva? 33

División entrenamiento / prueba 34

¿Qué función cumple stratify=y en esta división? 34

Decision Tree 34

¿Qué ventaja tiene limitar la profundidad del árbol? 34

¿Qué errores observas en la matriz de confusión? 34

Random Forest 35

¿Qué cambia al pasar de un árbol individual a un conjunto de árboles? 35

¿Mejora alguna métrica? ¿Cuál y cuánto? 35

Support Vector Machine (SVM) 36

¿Por qué en este modelo usamos StandardScaler? 36

¿Qué resultado obtiene SVM frente a los modelos basados en árboles? 36

K-Nearest Neighbors (K-NN) 37

¿Qué representa k=5 en este algoritmo? 37

¿Por qué el escalamiento es especialmente importante cuando se calculan distancias? 37

Naive Bayes 38

¿Qué supuesto simplificador utiliza Naive Bayes respecto a las variables? 38

¿En qué métricas se comporta mejor o peor que los otros clasificadores? 38

Conclusiones 40

Referencias 43

Introducción

En este informe se analiza, desde un enfoque académico, el funcionamiento práctico de cinco algoritmos de clasificación de *machine learning*: Árbol de Decisión (*Decision Tree*), Random Forest, Máquinas de Soporte Vectorial (*Support Vector Machine*, SVM), K-Vecinos más Cercanos (*K-Nearest Neighbors*, K-NN) y Naive Bayes Gaussiano (*GaussianNB*). El propósito no es probar ni calificar los modelos para determinar cuál es el mejor, sino que quien redacta el informe sea capaz de observar e interpretar el comportamiento de cada algoritmo durante su ejecución: cómo aprende de los datos, cómo toma decisiones, cómo responde a distintos parámetros y qué tipo de errores comete.

Para ello, los cinco algoritmos se ejecutan bajo las mismas condiciones, es decir, con el mismo conjunto de datos, la misma preparación y partición entre entrenamiento y prueba. Esto permite que las diferencias observadas se deban al funcionamiento propio de cada algoritmo y no a factores externos. Cada uno representa una forma distinta de abordar la clasificación. El Árbol de Decisión divide los datos mediante reglas jerárquicas fáciles de interpretar. Random Forest combina muchos árboles para reducir el sobreajuste y ganar estabilidad. SVM busca el límite de separación con el mayor margen entre clases. K-NN clasifica según la cercanía de cada dato a sus vecinos, sin construir un modelo explícito. Naive Bayes se basa en probabilidades y en el supuesto de independencia entre variables.

Durante la ejecución se registran las predicciones, la matriz de confusión, los tiempos de entrenamiento y predicción, y métricas básicas como exactitud, precisión, sensibilidad y F1. Estas métricas no se usan para declarar un ganador, sino como apoyo para interpretar lo observado: la sensibilidad a la escala de los datos, el tipo de frontera de decisión, la interpretabilidad, la tendencia al sobreajuste y el costo computacional. Finalmente, las

conclusiones se redactan a partir de la observación del proceso y no solo del desempeño numérico, con el fin de fortalecer la comprensión conceptual y práctica de cada algoritmo.

Caso de trabajo

Para observar el comportamiento de los cinco algoritmos se utiliza el conjunto de datos Breast Cancer Wisconsin (Diagnostic), conocido como WDBC, disponible directamente en scikit-learn. Es un conjunto clásico en aprendizaje automático y se emplea con frecuencia en contextos académicos por su tamaño manejable y su calidad. Contiene mediciones calculadas a partir de imágenes digitalizadas de muestras de tejido mamario y una etiqueta de diagnóstico. El problema es de clasificación binaria: determinar si un tumor es benigno o maligno a partir de las características de los núcleos celulares.

Para esta práctica, la variable objetivo se reorganiza de forma que 1 = caso maligno y 0 = caso benigno. Esto invierte la codificación original de scikit-learn, donde 0 corresponde a maligno y 1 a benigno. El cambio se realiza para que la clase de interés, el caso maligno, sea la clase positiva, lo que facilita la lectura de la matriz de confusión y de métricas como la precisión, la sensibilidad y el F1, que por defecto se calculan sobre la clase 1.

Este caso es adecuado para el informe porque permite observar con claridad varios aspectos que diferencian a los algoritmos. Las variables tienen escalas muy distintas (el área se mide en cientos o miles, mientras que la suavidad o la simetría están por debajo de 1), lo que hace visible cómo SVM y K-NN, que dependen de distancias, se ven afectados por la escala, mientras que el Árbol de Decisión y Random Forest no. Además, varias variables, como el radio, el perímetro y el área, están fuertemente correlacionadas, lo que contradice el supuesto de independencia de Naive Bayes y permite observar cómo responde el algoritmo ante ese incumplimiento. La proporción entre las clases, que no es equilibrada, obliga a mirar más allá de la exactitud global y a revisar la matriz de confusión. Por último, los errores tienen un costo

asimétrico: clasificar un tumor maligno como benigno (falso negativo) es mucho más grave que el error contrario, por lo que se presta especial atención a la sensibilidad de la clase maligna.

Los cinco algoritmos se ejecutan sobre los mismos datos, con una partición única entre entrenamiento y prueba, estratificada para conservar la proporción de las clases y con una semilla aleatoria fija para que los resultados sean reproducibles. Cuando el algoritmo lo requiere, como en SVM y K-NN, se estandarizan las variables. Este caso se utiliza únicamente con fines académicos y de observación; los modelos resultantes no deben interpretarse como herramientas de diagnóstico clínico.

Reglas Comunes

Para este ejercicio se establecieron reglas que garantizan que las condiciones sean idénticas para los cinco algoritmos. De este modo, los resultados son íntegros y reproducibles, y permiten comparar de manera objetiva las diferencias y similitudes en el funcionamiento de cada uno. Las reglas son las siguientes:

Misma división de datos

Los cinco modelos utilizan exactamente la misma partición entre entrenamiento y prueba. Así, cualquier diferencia en los resultados se debe al algoritmo y no a los datos con los que fue entrenado o evaluado.

Proporción 80/20 con partición estratificada.

Se usa el 80 % de los datos para entrenamiento y el 20 % para prueba, con random_state=42 y stratify=y. La semilla fija asegura que cualquier persona que repita el ejercicio obtenga la misma división, y la estratificación conserva en ambos conjuntos la proporción original entre casos benignos y malignos.

Conjunto de prueba intocable.

El conjunto de prueba no se modifica una vez iniciada la comparación. Esto evita que información de la evaluación influya en el entrenamiento o en la elección de parámetros (fuga de datos, o *data leakage*), lo que inflaría artificialmente los resultados.

Escalamiento solo donde es necesario.

Se aplica escalamiento a los algoritmos sensibles a la escala de las variables, SVM y K-NN, porque ambos dependen de distancias y una variable con valores grandes, como el área, dominaría sobre las demás. El escalador se ajusta únicamente con los datos de entrenamiento y

luego se aplica al conjunto de prueba, para no filtrar información. El Árbol de Decisión, Random Forest y Naive Bayes se ejecutan sin escalar, ya que su funcionamiento no depende de la magnitud de las variables.

Mismas métricas para todos.

Todos los modelos se evalúan con *Accuracy*, *Precision*, *Recall*, *F1-score* y matriz de confusión. Usar el mismo conjunto de métricas permite una comparación justa y muestra aspectos distintos del comportamiento: la exactitud global, la confiabilidad de las predicciones positivas, la capacidad de detectar los casos malignos y el equilibrio entre ambas.

No elegir el modelo solo por Accuracy.

La exactitud global puede ocultar errores importantes, sobre todo cuando las clases no están equilibradas y los errores no tienen el mismo costo. En este problema, un falso negativo (un tumor maligno clasificado como benigno) es mucho más grave que un falso positivo, por lo que se presta especial atención al *Recall* de la clase maligna. Por ello, la interpretación de los resultados se apoya en el análisis de los errores en la matriz de confusión y en el contexto del problema, y no en una única cifra.

Estas reglas permiten que el foco del informe esté en observar y comprender el comportamiento de cada algoritmo, y no en declarar un ganador.

Preparación de los datos

Para facilitar la comprensión del ejercicio, el código se dividió en varias secciones. Cada sección cumple una función específica dentro del flujo de trabajo, de modo que quien lea el informe pueda seguir paso a paso lo que ocurre, desde la carga de los datos hasta la interpretación de los resultados. Esta organización también permite identificar con claridad en qué punto se aplica cada una de las reglas del experimento y facilita repetir o modificar el ejercicio sin perder el hilo de lo que se hace.

Importación de pandas y data set

En esta parte se importa *pandas* con el comando import pandas as pd. Esta librería es fundamental porque permite limpiar, manipular y analizar datos de forma tabular (en filas y columnas) de manera rápida y sencilla. Su estructura principal, el *DataFrame*, funciona de manera similar a una hoja de cálculo y facilita tareas como seleccionar columnas, revisar estadísticas descriptivas, contar valores y detectar datos faltantes.

También se importa el conjunto de datos con el comando from sklearn.datasets import load_breast_cancer. Esta instrucción carga la función load_breast_cancer desde el módulo datasets de *scikit-learn*, la cual permite acceder directamente al conjunto *Breast Cancer Wisconsin (Diagnostic)* sin necesidad de descargar archivos externos ni conectarse a una fuente de datos. Al ejecutar la función, se obtiene un objeto que contiene las variables predictoras, la variable objetivo, los nombres de las características y una descripción del conjunto. Esto hace que el ejercicio sea fácil de reproducir, ya que cualquier persona que tenga *scikit-learn* instalado puede obtener exactamente los mismos datos.

En esta celda únicamente se cargan las herramientas y el conjunto de datos que se usarán en el resto del ejercicio; todavía no se realiza ningún análisis ni se entrena ningún modelo. Importar estos recursos al inicio deja claro desde el comienzo qué se utiliza y facilita la lectura y el mantenimiento del código.

Código 1

Celda de importación de datos

```
from sklearn. datasets import load_breast_cancer.
import pandas as pd
```

Carga del dataset en un Dataframe

Una vez cargada la función del conjunto de datos, se debe convertir su contenido en un *DataFrame* para que los datos sean más manejables y fáciles de analizar y limpiar. Al tener formato de tabla, los datos se vuelven más interactivos: se pueden filtrar, seleccionar por columnas, resumir con estadísticas descriptivas y graficar con mayor facilidad.

Esto se logra con el comando data = load_breast_cancer(as_frame=True). El parámetro as_frame=True le indica a *scikit-learn* que entregue los datos como objetos de *pandas* (un *DataFrame* para las variables predictoras y una *Series* para la variable objetivo) en lugar de arreglos de *NumPy*, que no tienen nombres de columnas visibles. El resultado se guarda en la variable data, que funciona como un contenedor con todos los componentes del conjunto.

Después, a partir de esa variable, se ejecuta el comando X = data.data.copy(), que permite extraer únicamente las variables predictoras, es decir, las 30 mediciones de los núcleos celulares, y guardarlas en X. El método .copy() crea una copia independiente de los datos originales, de modo que cualquier modificación posterior sobre X (como una transformación o una limpieza)

no altera el conjunto original almacenado en data. Así se conserva siempre una versión intacta a la cual volver si es necesario.

Por convención, X se escribe en mayúscula porque representa una matriz de variables (varias columnas), mientras que la variable objetivo se escribe en minúscula como y, ya que es un único vector.

En esta celda se transforma el conjunto de datos a un formato tabular y se separan las variables predictoras en X, dejando la información lista para la siguiente etapa: definir la variable objetivo.

Código 2

Conversión del dataset en dataframe

```
data = load_breast_cancer(as_frame=True)
x = data.data.copy()
```

Conversión la clase positiva en "Maligno"

Con el *DataFrame* ya creado, se define la variable objetivo y con el comando y = (data.target == 0).astype(int). Esta línea reorganiza la etiqueta original de *scikit-learn*, donde 0 corresponde a maligno y 1 a benigno, para que en este ejercicio 1 represente el caso maligno y 0 el caso benigno. El proceso ocurre en dos pasos: primero, la comparación data.target == 0 genera una serie de valores verdaderos (True) para los casos malignos y falsos (False) para los benignos; luego, .astype(int) convierte esos valores lógicos en números enteros (True pasa a ser 1 y False pasa a ser 0). Con esto, la clase de interés, el tumor maligno, queda como clase positiva, lo que facilita la lectura de la matriz de confusión y de métricas como *Precision*, *Recall* y *F1-score*.

A continuación, el código ejecuta dos instrucciones de verificación. print(X.shape) muestra las dimensiones de la matriz de variables predictoras, es decir, el número de registros (filas) y el número de variables (columnas), lo que permite confirmar que los datos se cargaron completos: 569 filas y 30 columnas. Por su parte, print(y.value_counts()) cuenta cuántos casos hay en cada clase, con lo que se comprueba que la conversión se hizo correctamente y se observa la distribución: 357 casos benignos (0) y 212 malignos (1). Esta revisión es importante porque confirma que las clases no están perfectamente equilibradas, un aspecto que justifica el uso de la estratificación en la división de los datos y el análisis de métricas más allá de la exactitud.

En esta celda se define la variable objetivo con la codificación deseada y se verifica que los datos y las clases correspondan con lo esperado antes de continuar con la división entre entrenamiento y prueba.

Código 3

Conversion la clase positiva en "Maligno"

```
y = (data. target == 0). astype(int)
print (x. shape)
print (y.value_counts ())
```

División de datos

Tras este pequeño proceso de ETL (extraer, transformar y cargar), se realiza la separación de los datos en dos subconjuntos: el de entrenamiento (train), que corresponde al 80 % del total de los datos, y el de prueba (test), que corresponde al 20 % restante. El conjunto de entrenamiento es el que usa cada algoritmo para aprender los patrones de los

datos, mientras que el de prueba se reserva para evaluar qué tan bien generaliza el modelo con casos que nunca ha visto.

Para ello se importa la función train_test_split del módulo model_selection de scikit-learn y se ejecuta el siguiente comando:

Código 4

```
from sklearn. model_selection import train_test_split.

X_train, X_test, y_train, y_test = train_test_split (
    x, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

La función devuelve cuatro objetos: X_train y y_train (variables predictoras y etiquetas de entrenamiento) y X_test y y_test (variables predictoras y etiquetas de prueba). Los parámetros utilizados cumplen cada uno una función concreta:

test_size=0.2: indica que el 20 % de los datos se destina a la prueba y, por lo tanto, el 80 % restante al entrenamiento. Con 569 registros, esto equivale aproximadamente a 455 para entrenamiento y 114 para prueba.

random_state=42: fija la semilla aleatoria. La división se hace al azar, pero con esta semilla el resultado es siempre el mismo, de modo que cualquier persona que repita el ejercicio obtiene exactamente la misma partición y los resultados son reproducibles.

stratify=y: hace que la división sea estratificada, es decir, que tanto el conjunto de entrenamiento como el de prueba conserven la misma proporción de casos benignos y malignos que tiene el conjunto original (cerca de 63 % y 37 %). Esto es importante

porque las clases no están equilibradas y, sin estratificar, el azar podría dejar el conjunto de prueba con muy pocos casos malignos, lo que distorsionaría la evaluación.

Esta división se hace una sola vez y se reutiliza para los cinco algoritmos, de modo que todos se entrenen y evalúen con exactamente los mismos datos. A partir de este punto, el conjunto de prueba no se modifica, para evitar la fuga de información (data leakage) y garantizar que la comparación sea objetiva.

Función común de evaluación

Antes de implementar los algoritmos se define una función que se reutiliza para los cinco modelos. Esto cumple la regla del experimento de evaluar todos con las mismas métricas: como la medición es siempre el mismo código, ningún algoritmo puede quedar favorecido por la forma de calcularla.

Primero se importan desde el módulo metrics de *scikit-learn* las funciones accuracy_score, precision_score, recall_score, f1_score y confusion_matrix. Luego se define la función evaluar_modelo, que recibe cuatro argumentos: el nombre del modelo (para identificarlo al imprimir), el modelo ya entrenado, los datos de evaluación (X_eval) y las etiquetas reales (y_eval). Dentro de la función, modelo.predict(X_eval) genera las predicciones sobre el conjunto de prueba y se comparan con las etiquetas reales para calcular las métricas, que se guardan en un diccionario. Ese diccionario se devuelve para poder construir después una tabla comparativa. La función también imprime el nombre del modelo y su matriz de confusión.

Las métricas se interpretan así:

Accuracy

proporción de predicciones correctas sobre el total.

Precision

De los casos que el modelo clasificó en una clase, cuántos realmente pertenecían a ella.

Recall

De los casos que realmente pertenecían a una clase, cuántos logró detectar el modelo.

F1-score

media armónica entre *Precision* y *Recall*, que resume el equilibrio entre ambas.

Matriz de confusión

tabla que cruza lo real con lo predicho. En este ejercicio la primera fila corresponde a los casos benignos reales y la segunda a los malignos reales. De izquierda a derecha y de arriba abajo se leen los verdaderos negativos, los falsos positivos, los falsos negativos y los verdaderos positivos. El **falso negativo** (un tumor maligno clasificado como benigno) es el error más grave en este contexto.

Las tres métricas de *Precision*, *Recall* y *F1-score* se calculan con average='weighted'. Con este parámetro la métrica se calcula por separado para cada clase y luego se promedia ponderando por el número de casos de cada una. Es importante notar que, al hacerlo así, el *Recall* ponderado coincide siempre con el *Accuracy*, y que las métricas no se refieren específicamente a la clase maligna. Por eso, además de estos valores, la interpretación se apoya en la matriz de confusión, donde sí se ven los falsos negativos de cada algoritmo.

Código 5

Función común de evaluación

```
      from sklearn. metrics import (accuracy_score, precision_score,
recall_score,
                                    f1_score, confusion_matrix)

      def evaluar_modelo(nombre, modelo, X_eval, y_eval):
          pred = modelo. predict(X_eval)

          resultados = {
              'modelo': nombre,
              'accuracy': accuracy_score(y_eval, pred),
              'precision': precision_score (y_eval, pred, average='weighted'),
              'recall': recall_score (y_eval, pred, average='weighted'),
              'f1': f1_score (y_eval, pred, average='weighted'),
              'confusion_matrix': confusion_matrix(y_eval, pred)
          }
          print(nombre)
          print(confusion_matrix(y_eval, pred))
          return resultados
```

En esta celda no se entrena ningún modelo. Solo se deja lista la herramienta de medición que se usará con cada algoritmo.

Implementación de los clasificadores

Árbol de Decisión (*Decision Tree*)

El Árbol de Decisión clasifica los datos mediante una secuencia de preguntas sobre las variables (por ejemplo, si el radio medio supera cierto valor), dividiendo los datos en ramas hasta llegar a una decisión. Su principal ventaja es la interpretabilidad, porque las reglas aprendidas pueden leerse directamente. Su debilidad es la tendencia al sobreajuste: si crece sin límite, memoriza los datos de entrenamiento.

Para controlarlo se usa max_depth=4, que limita el árbol a cuatro niveles de preguntas. Con random_state=42 se fija la semilla, porque el algoritmo tiene componentes aleatorios en el desempate entre variables. Este algoritmo no requiere escalamiento, ya que sus decisiones dependen del orden de los valores y no de su magnitud.

Código 6

Implementación del Árbol de Decisión

```
from sklearn. three import DecisionTreeClassifier.
modelo_dt = DecisionTreeClassifier (max_depth=4, random_state=42)
modelo_dt.fit (X_train, y_train)
res_dt = evaluar_modelo ('Decision Tree', modelo_dt, X_test, y_test)
print (
    f"Accuracy: res_dt['accuracy']:.4f} | "
    f"Precision: res_dt['precision']:.4f} | "
    f"Recall: res_dt['recall']:.4f} | "
    f"F1: res_dt['f1']:.4f”
)
```

Al ejecutarlo se obtiene una exactitud de 0.9123. La matriz de confusión muestra 70 benignos y 34 malignos bien clasificados, con 2 falsos positivos y 8 falsos negativos. Es el modelo con más tumores malignos no detectados, lo que lo hace el menos adecuado en este contexto, a pesar de ser el más fácil de interpretar.

Random Forest

Random Forest es un método de conjunto (*ensemble*) que entrena muchos árboles de decisión, cada uno con una muestra distinta de los datos y de las variables, y decide por votación.

Al combinar árboles distintos se compensan los errores individuales, lo que reduce el sobreajuste y da más estabilidad que un solo árbol.

Se usan n_estimators=100, es decir, 100 árboles, y max_depth=4, la misma profundidad máxima del árbol individual, para que la comparación entre ambos sea más directa. random_state=42 hace reproducibles los resultados. Tampoco requiere escalamiento.

Código 7

Implementación de Random Forest

```
from sklearn. ensemble import RandomForestClassifier.
modelo_rf = RandomForestClassifier (n_estimators=100, max_depth=4,
random_state=42)
modelo_rf.fit (X_train, y_train)
res_rf = evaluar_modelo ('Random Forest', modelo_rf, X_test, y_test)
print (
    f"Accuracy: res_rf['accuracy']:.4f} | "
    f"Precision: res_rf['precision']:.4f} | "
    f"Recall: res_rf['recall']:.4f} | "
    f"F1: res_rf['f1']:.4f”
)
```

La exactitud sube a 0.9649, y la matriz de confusión muestra 72 benignos y 38 malignos bien clasificados, sin falsos positivos y con 4 falsos negativos. Con la misma profundidad que el árbol individual, el conjunto reduce los falsos negativos de 8 a 4, lo que ilustra el efecto de combinar varios árboles.

Máquina de Soporte Vectorial (Support Vector Machine, SVM)

SVM busca el límite de separación (hiperplano) entre las dos clases que deje el mayor margen posible respecto a los casos más cercanos de cada una. Con el núcleo rbf (función de base radial) puede construir fronteras curvas, lo que le permite separar datos que no son separables con una línea recta.

Este algoritmo depende de distancias, por lo que es sensible a la escala de las variables: el área, con valores de cientos o miles, dominaría sobre la suavidad, con valores menores a 1. Por eso se usa un *pipeline* (make_pipeline) que encadena un StandardScaler, que estandariza las variables a media 0 y desviación 1, con el clasificador SVC. Al ejecutar fit, el escalador se ajusta solo con los datos de entrenamiento, y en la predicción aplica esa misma transformación al conjunto de prueba, lo que evita la fuga de información. Los parámetros son kernel='rbf', C=1.0, que controla qué tanto se penalizan los errores de entrenamiento, y gamma='scale', que ajusta automáticamente el alcance de cada punto según los datos.

Código 8

Implementación de la Máquina de Soporte Vectorial (SVM)

```
        from sklearn. pipeline import makes_pipeline.
        from sklearn. preprocessing import StandardScaler
        from sklearn.svm import SVC
        modelo_svm = make_pipeline (StandardScaler (), SVC (kernel='rbf', C=1.0,
gamma='scale'))
        modelo_svm.fit (X_train, y_train)
        res_svm = evaluar_modelo('SVM', modelo_svm, X_test, y_test)
        print (
            f"Accuracy: res_svm['accuracy']:.4f} | "
```

```
            f"Precision: res_svm['precision']:.4f} | "
            f"Recall: res_svm['recall']:.4f} | "
            f"F1: res_svm['f1']:.4f”
        )
```

SVM obtiene la mayor exactitud, 0.9737. Su matriz de confusión muestra 72 benignos y 39 malignos bien clasificados, sin falsos positivos y con solo 3 falsos negativos, el menor número de los cinco algoritmos.

K-Vecinos más Cercanos (*K-Nearest Neighbors*, K-NN)

K-NN no construye un modelo explícito durante el entrenamiento. Simplemente almacena los datos y, para clasificar un caso nuevo, busca los *k* casos de entrenamiento más cercanos y asigna la clase que predomine entre ellos. Por eso se le llama algoritmo “perezoso” (*lazy*): el trabajo se hace al predecir y no al entrenar.

Al basarse por completo en distancias, es muy sensible a la escala, y se usa el mismo *pipeline* con StandardScaler que en SVM. El parámetro n_neighbors=5 indica que cada caso se clasifica según sus cinco vecinos más cercanos.

Código 9

Implementación de K-Vecinos más Cercanos (K-NN)

```
        from Sklenar. neighbors import KNeighborsClassifier.


        modelo_knn = make_pipeline (StandardScaler (),
KNeighborsClassifier(n_neighbors=5))
        modelo_knn.fit (X_train, y_train)
        res_knn = evaluar_modelo ('KNN', modelo_knn, X_test, y_test)
        print (
```

```
        f"Accuracy: res_knn['accuracy']:.4f} | "
        f"Precision: res_knn['precision']:.4f} | "
        f"Recall: res_knn['recall']:.4f} | "
        f"F1: res_knn['f1']:.4f”
    )
```

K-NN alcanza una exactitud de 0.9561. La matriz de confusión muestra 71 benignos y 38 malignos bien clasificados, con 1 falso positivo y 4 falsos negativos, un resultado muy cercano al de Random Forest sin haber entrenado ningún modelo propiamente dicho.

Naive Bayes Gaussiano (GaussianNB)

Naive Bayes aplica el teorema de Bayes para calcular la probabilidad de que un caso pertenezca a cada clase, y asume que las variables son independientes entre sí y que, dentro de cada clase, siguen una distribución normal (gaussiana). Es muy rápido y sencillo, pero el supuesto de independencia casi nunca se cumple por completo. En este conjunto de datos se incumple de forma evidente, porque el radio, el perímetro y el área están fuertemente correlacionados.

No requiere escalamiento, ya que calcula la media y la desviación de cada variable por separado y no usa distancias. Se usa con los parámetros por defecto, sin hiperparámetros que ajustar.

Código 10

Implementación de Naive Bayes Gaussiano

```
from sklearn. naive_bayes import GaussianNB.
model_on_nb = GaussianNB ()
modelo_nb.fit (X_train, y_train)
```

```
res_nb = evaluar_modelo ('Naive Bayes', modelo_nb, X_test, y_test)
print (
    f"Accuracy: res_nb['accuracy']:.4f} | "
    f"Precision: res_nb['precision']:.4f} | "
    f"Recall: res_nb['recall']:.4f} | "
    f"F1: res_nb['f1']:.4f”
)
```

La exactitud es de 0.9386. La matriz de confusión muestra 72 benignos y 35 malignos bien clasificados, sin falsos positivos y con 7 falsos negativos. Pese a incumplir el supuesto de independencia, rinde mejor que el Árbol de Decisión, aunque deja pasar más tumores malignos que Random Forest, SVM y K-NN.

Comparación para la toma de decisiones

Tabla 1

Tabla de comparación de los 5 modelos elegidos

Modelo	Accuracy	Presision	Recall	F1	Observación
Decsion Tree	0.9123	0.9148	0.9123	0.9107	Su matriz de confusión es: 72 verdaderos negativos 2 falsos positivos 8 falsos negativos 34 verdaderos positivos

Random forest	0.9649	0.9668	0.9649	0.9645	Su matriz de confusión es: 72 verdaderos negativos 0 falsos positivos 4 falsos negativos 28 verdaderos positivos
SVM	0.9737	0.9747	0.9737	0.9735	Su matriz de confusión es: 72 verdaderos negativos 0 falsos positivos 3 falsos negativos 39 verdaderos positivos
K-NN	0.9561	0.9569	0.9561	0.9558	Su matriz de confusión es: 71 verdaderos negativos 1 falsos positivos 4 falsos negativos 38 verdaderos positivos
Native Bayes	0.9386	0.9440	0.9386	0.9372	Su matriz de confusión es: 72 verdaderos negativos 0 falsos positivos 7 falsos negativos 35 verdaderos positivos

Preguntas de análisis

¿Qué modelo obtuvo la mayor Accuracy?

Según los datos recolectados, el modelo con mejor desempeño en términos de *accuracy* fue la Máquina de Vectores de Soporte (SVM, por sus siglas en inglés), con un valor de 0,9737, es decir, un 97,37 %. Este resultado indica que el modelo clasifica correctamente la gran mayoría de los casos, por lo que se selecciona para realizar las predicciones del sistema.

¿Qué modelo obtuvo el mayor Recall para la clase maligna?

Asimismo, se observó que en todos los modelos el valor del accuracy coincide exactamente con el del recall al menos en los primeros dígitos. Esto se debe a que el recall se calculó mediante promedio ponderado (weighted average), que pondera cada clase según su

número de muestras y, en consecuencia, equivale al accuracy. Por lo tanto, el modelo con mayor recall es la Máquina de Vectores de Soporte (SVM), que también obtuvo el mayor accuracy.

¿Qué modelo produjo menos falsos negativos?

Al comparar las matrices de confusión de los modelos, se observó que la Máquina de Vectores de Soporte (SVM) es el modelo con menos falsos negativos, con un total de 3. Esto significa que SVM es el que menos cas os malignos clasifica erróneamente como benignos, lo que lo convierte en el modelo más confiable para la detección de la clase maligna.

¿Coincide el modelo con mayor Accuracy con el modelo que seleccionarías? Explica.

El modelo con mayor *Accuracy* fue el SVM (0.9737), que además presentó el menor número de falsos negativos (3). Sin embargo, la diferencia respecto a Random Forest (0.9649, con 4 falsos negativos) es de un solo caso sobre 114 registros de prueba, por lo que no es concluyente. Se selecciona Random Forest porque combina un desempeño cercano al mejor con ausencia de falsos positivos, no requiere escalamiento de las variables y permite analizar la importancia de cada característica, lo que facilita explicar sus decisiones al personal clínico. Esta elección no se basa únicamente en el *Accuracy*, sino en el contexto del problema: en un diagnóstico, dejar pasar un tumor maligno es el error más costoso, y Random Forest redujo ese error a la mitad respecto al Árbol de Decisión individual (de 8 a 4).

Si el costo de no detectar un caso maligno fuera especialmente alto, ¿qué métrica priorizarías y por qué?

Si el costo de no detectar un caso maligno fuera especialmente alto, se priorizaría el *Recall* de la clase maligna, ya que esta métrica mide la proporción de tumores malignos que el modelo logra detectar y penaliza directamente los falsos negativos. La *Precision*, en cambio,

mide la confiabilidad de las alarmas y no refleja los casos que el modelo deja pasar: en este ejercicio, Naive Bayes alcanzó una *Precision* de 1.000 para la clase maligna y aun así produjo 7 falsos negativos. Por ello, en un contexto de diagnóstico, el *Recall* es el criterio más adecuado, complementado con el *F1-score* para vigilar que no aumenten en exceso los falsos positivos.

¿Cuál de los modelos es más sencillo de explicar a una persona no técnica?

El modelo más sencillo de explicar a una persona no técnica es el Árbol de Decisión, ya que su funcionamiento equivale a un cuestionario de preguntas de sí o no que puede representarse gráficamente y seguirse paso a paso hasta el diagnóstico. Esta transparencia permite justificar cada predicción y que un especialista revise si las reglas aprendidas tienen sentido clínico. Sin embargo, en este ejercicio fue también el modelo con mayor número de falsos negativos (8), por lo que su interpretabilidad se obtiene a costa de una menor capacidad para detectar casos malignos.

¿Qué diferencias observaste entre los algoritmos que requieren escalamiento y los que no?

Se observó que los algoritmos basados en distancias, SVM y K-NN, requieren escalamiento de las variables, porque clasifican según qué tan cerca o lejos están los casos entre sí y, sin estandarizar, las variables de mayor magnitud, como el área o el perímetro, dominarían el cálculo. En cambio, el Árbol de Decisión, Random Forest y Naive Bayes no lo necesitan: los dos primeros toman decisiones comparando cada variable con un umbral, y el tercero modela cada variable de forma independiente, por lo que ninguno se ve afectado por la magnitud de los valores. Para evitar fuga de información, el escalamiento se ajustó únicamente con el conjunto de entrenamiento mediante un *pipeline*. Cabe aclarar que en este ejercicio no se ejecutaron SVM y K-NN sin escalar, por lo que la comparación se basa en el fundamento del funcionamiento de cada algoritmo y no en una medición directa del efecto.

Selecciona un modelo final y justifica tu decisión usando al menos dos métricas y una consideración del contexto

Se selecciona la Máquina de Soporte Vectorial (SVM) como modelo final. Este algoritmo obtuvo el mayor *Accuracy* (0.9737) y el mayor *Recall* para la clase maligna (aproximadamente 0.93), al producir solo 3 falsos negativos sobre 42 tumores malignos del conjunto de prueba, el menor número entre los cinco algoritmos. Además, alcanzó una *Precision* de 1.000 para la clase maligna, es decir, ninguno de los casos que marcó como malignos fue una falsa alarma.

La decisión no se basa únicamente en el *Accuracy*, sino en el contexto del problema. En el diagnóstico temprano de cáncer de mama, el error más costoso es el falso negativo, pues un tumor maligno no detectado puede retrasar el tratamiento. Por ello se priorizó el *Recall* de la clase maligna, y el SVM fue el modelo que mejor lo cumplió. Su limitación es la interpretabilidad: al usar un núcleo rbf, la frontera de decisión es curva y difícil de explicar a personal no técnico, a diferencia del Árbol de Decisión. Por esta razón, el modelo se concibe como una herramienta de apoyo a la decisión clínica y no como un sustituto del criterio médico.

Cabe señalar que la ventaja sobre Random Forest es de un solo caso sobre 114 registros de prueba, por lo que no es concluyente. Antes de un uso real se requeriría validación cruzada, ajuste de hiperparámetros y evaluación con datos independientes.

Respuestas de preguntas

Preparación del caso y datos

¿Cuántas observaciones y variables contiene el conjunto de datos?

El conjunto de datos contiene 569 observaciones (filas que corresponden a muestras de pacientes) y 30 variables predictoras (columnas numéricas con características morfológicas del núcleo celular), además de la variable objetivo de diagnóstico.

¿La variable objetivo está balanceada? Explica brevemente lo que observas.

No está completamente balanceada, presenta un desbalance moderado.Al analizar la distribución de las clases observamos que la clase 0 (casos benignos) representa el 62.74% del total con 357 muestras, mientras que la clase 1 (casos malignos) abarca el 37.26% con 212 muestras.Aunque no se trata de un desbalance extremo, esta diferencia implica que la métrica de exactitud (Accuracy) por sí sola no será suficiente para evaluar el modelo, ya que un algoritmo simplista que predijera siempre "benigno" alcanzaría casi un 63% de aciertos sin detectar un solo caso de cáncer.

¿Por qué es importante definir con claridad qué representa la clase positiva?

Es fundamental por tres razones principales:Gestión del riesgo clínico: Permite enfocar la optimización del modelo en reducir los Falsos Negativos (diagnosticar como benigno a un paciente que padece un tumor maligno), lo cual representa el error de mayor costo humano y médico.Cálculo correcto de métricas: Las funciones de evaluación en bibliotecas como scikit-learn calculan por defecto la precisión y la sensibilidad tomando como referencia el valor 1. Si no se define claramente cuál es la clase de interés, los resultados numéricos se interpretan de forma errónea.Alineación con el equipo: Garantiza que tanto los desarrolladores como el

personal médico interpreten correctamente los porcentajes de probabilidad que devuelva el sistema predictivo.

División entrenamiento / prueba

¿Qué función cumple stratify=y en esta división?

mantiene la misma proporción de clases en los conjuntos de entrenamiento y prueba que la que hay en los datos originales.

Sirve para evitar que una clase quede subrepresentada o ausente en el train o el test, algo fundamental en problemas de clasificación (especialmente si los datos están desbalanceados).

Decision Tree

¿Qué ventaja tiene limitar la profundidad del árbol?

Limitar la profundidad del árbol de decisión (max_depth=4) reduce el riesgo de sobreajuste, ya que impide que el modelo memorice el ruido y los casos atípicos del conjunto de entrenamiento y favorece que generalice mejor a datos nuevos. Además, mantiene el árbol lo bastante pequeño como para ser interpretado visualmente, y lo hace más estable frente a pequeñas variaciones en los datos. La contrapartida es el riesgo de subajuste: si el límite es demasiado bajo, el modelo no captura la complejidad del problema. En este ejercicio se fijó la profundidad en 4 de forma preventiva, sin compararla con otros valores, por lo que no se midió directamente su efecto sobre el desempeño.

¿Qué errores observas en la matriz de confusión?

La matriz de confusión del Árbol de Decisión muestra 70 verdaderos negativos, 2 falsos positivos, 8 falsos negativos y 34 verdaderos positivos, es decir, 10 errores sobre 114 casos de

prueba. El error más relevante es el de los 8 falsos negativos, tumores malignos clasificados como benignos, que equivalen a casi 19 % de los casos malignos del conjunto de prueba (*Recall* de la clase maligna ≈ 0.81). Los 2 falsos positivos representan falsas alarmas de menor gravedad clínica, pues solo implicarían estudios adicionales. Aunque el árbol es confiable cuando clasifica un caso como maligno (*Precision* ≈ 0.94), deja pasar una proporción importante de tumores malignos, lo que lo hace el menos adecuado de los cinco algoritmos cuando el costo de no detectar un caso es alto. Estos resultados provienen de una única partición de prueba de 114 registros y deben interpretarse con cautela.

Random Forest

¿Qué cambia al pasar de un árbol individual a un conjunto de árboles?

Al pasar de un árbol individual a un conjunto de árboles (Random Forest) cambia la forma de decidir: cada uno de los 100 árboles se entrena con una muestra aleatoria de los datos y considera subconjuntos aleatorios de variables en cada división, y la clasificación final se obtiene por votación. Esto reduce la varianza y el sobreajuste, pues los errores de unos árboles se compensan con los aciertos de otros. En este ejercicio, con la misma profundidad máxima (4), el *Accuracy* aumentó de 0.9123 a 0.9649, los falsos negativos pasaron de 8 a 4 y los falsos positivos de 2 a 0. A cambio, se pierde interpretabilidad, porque ya no es posible seguir una única ruta de decisión, y aumenta el costo computacional. Estos resultados corresponden a una única partición de prueba y deben interpretarse con cautela.

¿Mejora alguna métrica? ¿Cuál y cuánto?

Random Forest mejoró todas las métricas respecto al Árbol de Decisión. El *Accuracy* aumentó de 0.9123 a 0.9649 (+0.0526) y el *F1-score* ponderado de 0.9107 a 0.9645 (+0.0538).

La mejora más relevante se observa en el *Recall* de la clase maligna, que pasó de aproximadamente 0.81 a 0.905 (cerca de 9.5 puntos porcentuales), al reducirse los falsos negativos de 8 a 4. Además, la *Precision* de la clase maligna alcanzó 1.000, pues se eliminaron los dos falsos positivos. Estas diferencias provienen de una única partición de prueba de 114 registros, por lo que deben interpretarse con cautela.

Support Vector Machine (SVM)

¿Por qué en este modelo usamos StandardScaler?

Se utilizó StandardScaler en el SVM porque este algoritmo clasifica a partir de distancias entre casos y es sensible a la escala de las variables. En este conjunto de datos, variables como el área y el perímetro tienen valores de cientos o miles, mientras que otras, como la suavidad o la simetría, son menores a 1; sin estandarizar, las primeras dominarían el cálculo y las demás tendrían una influencia mínima. El escalador transforma cada variable a media 0 y desviación estándar 1, y se integró en un *pipeline* para ajustarse únicamente con los datos de entrenamiento y aplicarse luego al conjunto de prueba, evitando la fuga de información. En este ejercicio no se ejecutó el SVM sin escalar, por lo que no se midió directamente el efecto del escalamiento sobre el desempeño.

¿Qué resultado obtiene SVM frente a los modelos basados en árboles?

El SVM obtuvo mejores resultados que los dos modelos basados en árboles. Su *Accuracy* fue de 0.9737, frente a 0.9649 de Random Forest y 0.9123 del Árbol de Decisión, y presentó el menor número de falsos negativos (3, frente a 4 y 8, respectivamente), con lo que detectó 39 de los 42 tumores malignos del conjunto de prueba. La ventaja sobre el Árbol de Decisión es amplia, mientras que la diferencia con Random Forest es de un solo caso sobre 114 registros, por

lo que no es concluyente. A cambio, los modelos de árboles ofrecen mayor interpretabilidad: el Árbol de Decisión puede representarse como una secuencia de reglas y Random Forest permite estimar la importancia de las variables, mientras que la frontera curva del SVM es más difícil de explicar a personal no técnico. Estos resultados corresponden a una única partición de prueba y deben interpretarse con cautela.

K-Nearest Neighbors (K-NN)

¿Qué representa k=5 en este algoritmo?

El parámetro k=5 indica el número de vecinos más cercanos que el algoritmo K-NN consulta para clasificar un caso nuevo: se identifican los cinco casos de entrenamiento más próximos y se asigna la clase que predomine entre ellos mediante votación. Se usa un valor impar para evitar empates entre las dos clases. Un k pequeño hace al modelo más sensible al ruido y propenso al sobreajuste, mientras que un k grande suaviza la frontera de decisión y puede favorecer a la clase mayoritaria. En este ejercicio se empleó k=5, valor por defecto de *scikit-learn*, sin compararlo con otros, por lo que no se puede afirmar que sea el valor óptimo.

¿Por qué el escalamiento es especialmente importante cuando se calculan distancias?

El escalamiento es especialmente importante en los algoritmos que calculan distancias, como K-NN y SVM, porque la distancia combina las diferencias de todas las variables en sus unidades originales. En este conjunto de datos, variables como el área o el perímetro, con valores de cientos o miles, aportarían diferencias muchísimo mayores que variables como la suavidad o la simetría, menores a 1, de modo que estas últimas casi no influirían en la distancia aunque fueran igualmente informativas. Al estandarizar cada variable a media 0 y desviación estándar 1, todas contribuyen de forma comparable al cálculo. En cambio, los árboles de decisión, Random

Forest y Naive Bayes no combinan variables en una distancia, por lo que no requieren este paso. En este ejercicio no se ejecutaron K-NN ni SVM sin escalar, por lo que el efecto no se midió directamente.

Naive Bayes

¿Qué supuesto simplificador utiliza Naive Bayes respecto a las variables?

El supuesto simplificador de Naive Bayes es que las variables son independientes entre sí dentro de cada clase, y en la versión Gaussiana, que además siguen una distribución normal. Gracias a esto, la probabilidad conjunta se reduce al producto de las probabilidades de cada variable por separado, lo que hace al algoritmo muy rápido y sencillo. En este conjunto de datos el supuesto se incumple, pues variables como el radio, el perímetro y el área están fuertemente correlacionadas. Aun así, el modelo obtuvo un *Accuracy* de 0.9386, con 0 falsos positivos y 7 falsos negativos, lo que indica que puede rendir de forma aceptable pese a este incumplimiento, aunque dejó pasar más tumores malignos que Random Forest, SVM y K-NN. No se midió directamente si la correlación entre variables influyó en esos errores.

¿En qué métricas se comporta mejor o peor que los otros clasificadores?

Naive Bayes ocupó el cuarto lugar entre los cinco algoritmos en las métricas ponderadas (*Accuracy* 0.9386, *F1-score* 0.9372), superando únicamente al Árbol de Decisión y quedando por debajo de SVM, Random Forest y K-NN. Su mejor desempeño se observó en la *Precision* de la clase maligna (1.000), con 0 falsos positivos, igual que SVM y Random Forest. Su punto débil fue el *Recall* de la clase maligna (aproximadamente 0.83), con 7 falsos negativos, más del doble que el SVM (3). Esto indica que el modelo es muy conservador al clasificar un caso como maligno, pero deja pasar una proporción importante de tumores malignos, lo cual es desfavorable

cuando el costo de no detectarlos es alto. Estos resultados provienen de una única partición de prueba y deben interpretarse con cautela.

Conclusiones

Este informe evalúa el desempeño de cinco algoritmos de aprendizaje supervisado (Árbol de Decisión, Random Forest, Support Vector Machines, K-Nearest Neighbors y Naive Bayes Gaussiano) sobre el conjunto de datos *Breast Cancer Wisconsin (Diagnostic)*, bajo condiciones estrictamente controladas. Todos los modelos compartieron una partición estratificada 80/20 (random_state=42) con un conjunto de prueba inalterado de 114 registros (42 casos malignos y 72 benignos). Para los algoritmos basados en distancias (SVM y K-NN), el escalamiento de variables se estructuró dentro de un *pipeline* ajustado únicamente con los datos de entrenamiento, previniendo cualquier fuga de información (*data leakage*). Esta homogeneidad metodológica garantiza que las variaciones observadas respondan exclusivamente a los mecanismos propios de cada algoritmo y no a factores externos.

En términos de rendimiento global, el **SVM con kernel RBF** alcanzó los mejores resultados con un *Accuracy* de 0.9737, cometiendo solo 3 falsos negativos y ningún falso positivo. Le siguieron **Random Forest** (*Accuracy* de 0.9649 y 4 falsos negativos), **K-NN** (*Accuracy* de 0.9561, con 4 falsos negativos y 1 falso positivo), **Naive Bayes Gaussiano** (*Accuracy* de 0.9386 y 7 falsos negativos) y el **Árbol de Decisión** (*Accuracy* de 0.9123, con 8 falsos negativos y 2 falsos positivos). Aunque la diferencia entre SVM y Random Forest representa solo un caso de margen sobre la muestra de prueba y no resulta matemáticamente concluyente, la ventaja de SVM sobre el Árbol de Decisión individual es amplia, logrando identificar 39 de los 42 tumores malignos frente a solo 34 del árbol.

El análisis de los datos evidenció que el *Accuracy* por sí solo resulta una métrica insuficiente para la toma de decisiones clínicas. La configuración predeterminada de métricas ponderadas en el código hace que el *Recall* promedio coincida con el *Accuracy* global, ocultando

la capacidad real del modelo para detectar la clase crítica. En el diagnóstico médico, un falso negativo representa enviar a casa a un paciente con un tumor maligno no detectado, constituyendo el error de mayor costo y riesgo. Por ello, el **Recall de la clase maligna** debe ser el criterio prioritario de selección, ámbito donde el SVM demostró la mayor solidez (≈ 0.929) y el Árbol de Decisión la mayor debilidad (≈ 0.810). Esta necesidad de profundizar en el error se confirmó con Naive Bayes, que alcanzó una *Precision* perfecta de 1.000 en la clase maligna (cero falsos positivos) pero dejó pasar 7 tumores, demostrando que una alta precisión no compensa una baja sensibilidad en escenarios médicos.

Desde la perspectiva del comportamiento técnico de los algoritmos, se observaron claras diferencias operativas. SVM y K-NN requirieron una estandarización previa para evitar que características con magnitudes numéricas elevadas, como el área o el perímetro, ejercieran un dominio indebido sobre variables de menor escala como la suavidad o la simetría; en contraste, los modelos de árboles y Naive Bayes no exigieron dicha transformación. Por otra parte, la transición de un Árbol de Decisión individual (max_depth=4) a un ensemble de Random Forest con 100 árboles a la misma profundidad redujo los falsos negativos a la mitad (de 8 a 4) y eliminó los falsos positivos (de 2 a 0), ilustrando cómo la combinación por voto mayoritario mitiga drásticamente la varianza. Finalmente, Naive Bayes superó al Árbol de Decisión a pesar de violar el supuesto de independencia condicional entre atributos debido a la alta correlación entre radio, perímetro y área, aunque se mantuvo por debajo de los modelos más competitivos.

El estudio pone de manifiesto un compromiso directo entre el rendimiento del modelo y su nivel de interpretabilidad. El Árbol de Decisión destaca por ser el más intuitivo y fácil de explicar a personal sanitario no especializado, ya que su estructura equivale a una guía de preguntas binarias de sí o no; sin embargo, fue la alternativa que omitió más casos malignos. En

el extremo opuesto, la frontera de decisión compleja del SVM con núcleo rbf ofreció la mayor tasa de acierto y la menor cantidad de errores críticos, a costa de comportarse como una "caja negra" de difícil explicación técnica. Random Forest ofrece un punto de equilibrio intermedio al mantener una alta precisión y permitir la extracción del valor de importancia relativa de las variables. Por estas razones, se selecciona al **SVM como el modelo final**, concebido como una herramienta de soporte a la decisión clínica para complementar la evaluación profesional y nunca como un sustituto del criterio médico.

Estos resultados deben interpretarse con cautela debido a limitaciones metodológicas: provienen de un único split de prueba de 114 registros, no incluyeron una optimización sistemática de hiperparámetros (como ajustar el valor de k en K-NN o la profundidad máxima en los árboles) ni aislaron formalmente el impacto de omitir el escalamiento en los modelos basados en distancias. Para trabajos futuros, se recomienda aplicar validación cruzada k-fold, realizar una búsqueda en malla (*GridSearchCV*) para la calibración de parámetros, computar las métricas sobre la clase maligna de forma directa mediante la configuración average='binary' y evaluar la capacidad de generalización del sistema sobre conjuntos de datos externos e independientes antes de contemplar cualquier despliegue práctico. Se reitera que los modelos desarrollados poseen un fin estrictamente académico y no deben emplearse como herramientas de diagnóstico en entornos médicos reales.

Referencias

Anthropic. (2026). *Claude Sonnet 5.5* [Large language model]. https://claude.ai

McKinney, W. (2010). Data structures for statistical computing in Python. En S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56-61). https://doi.org/10.25080/Majora-92bf1922-00a

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.

Street, W. N., Wolberg, W. H., & Mangasarian, O. L. (1993). Nuclear feature extraction for breast tumor diagnosis. *Proceedings of SPIE, 1905*, 861-870. https://doi.org/10.1117/12.148698

Wolberg, W. H., Street, W. N., & Mangasarian, O. L. (1995). *Breast Cancer Wisconsin (Diagnostic)* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C5DW2B

Aplica sangría francesa (0,5 pulgadas) a cada referencia y ordénalas alfabéticamente, como están.